```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Employee Task Management · React demo</title>
  <!-- Load React and ReactDOM from CDN (for quick prototyping) -->
  <script crossorigin src="https://unpkg.com/react@18.2.0/umd/react.development.js"></script>
  <script crossorigin src="https://unpkg.com/react-dom@18.2.0/umd/react-dom.development.js"></script>
  <!-- Babel to transform JSX in the browser -->
  <script src="https://unpkg.com/@babel/standalone/babel.min.js"></script>
  <style>
    body {
      font-family: 'Segoe UI', Tahoma, Geneva, Verdana, sans-serif;
      background: #f4f7fb;
      display: flex;
      justify-content: center;
      align-items: center;
      min-height: 100vh;
      margin: 0;
      padding: 1rem;
    }
    .card {
      background: white;
      max-width: 550px;
      width: 100%;
      border-radius: 28px;
      box-shadow: 0 20px 35px -8px rgba(0,20,40,0.25);
      padding: 2rem 2rem 2.5rem 2rem;
      transition: all 0.2s ease;
    }
    h1 {
      font-size: 2rem;
      font-weight: 600;
      color: #1a2e3f;
      margin-top: 0;
      margin-bottom: 0.35rem;
      letter-spacing: -0.02em;
      border-left: 7px solid #3b82f6;
      padding-left: 1.25rem;
    }
    .subhead {
      color: #4b587c;
      margin-left: 1.9rem;
      margin-bottom: 2rem;
      font-size: 0.95rem;
      font-weight: 400;
    }
    .task-list {
      list-style: none;
      padding: 0;
      margin: 0;
      display: flex;
      flex-direction: column;
      gap: 0.85rem;
    }
    .task-item {
      background: #f9fafc;
      border-radius: 60px;
      display: flex;
      align-items: center;
      justify-content: space-between;
      padding: 0.6rem 0.6rem 0.6rem 1.6rem;
      border: 1px solid #e2e8f0;
      box-shadow: 0 2px 5px rgba(0,0,0,0.02);
      transition: all 0.15s;
    }
    .task-item:hover {
      background: #ffffff;
      border-color: #cbd5e1;
      box-shadow: 0 6px 14px rgba(0,30,60,0.08);
    }
    .task-info {
      display: flex;
      align-items: center;
      gap: 1rem;
    }
    .task-name {
      font-weight: 550;
      color: #0f1a2c;
      font-size: 1.1rem;
    }
    .status-badge {
      font-size: 0.75rem;
      font-weight: 600;
      padding: 0.25rem 0.8rem;
      border-radius: 40px;
      text-transform: uppercase;
      letter-spacing: 0.3px;
    }
    .status-done {
      background: #d1fae5;
      color: #0b5e42;
      border: 1px solid #a7f3d0;
    }
    .status-not-done {
      background: #fee2e2;
      color: #991b1b;
      border: 1px solid #fecaca;
    }
    .toggle-btn {
      background: white;
      border: 1.5px solid #d1d9e6;
      border-radius: 40px;
      padding: 0.45rem 1.4rem;
      font-size: 0.85rem;
      font-weight: 550;
      color: #25455b;
      cursor: pointer;
      transition: 0.1s ease;
      box-shadow: 0 2px 3px rgba(0,0,0,0.02);
    }
    .toggle-btn:hover {
      background: #eef2f8;
      border-color: #8ba0bc;
      color: #0b1e2e;
      transform: scale(0.98);
    }
```

```
    .toggle-btn:active {
      background: #d6e0eb;
    }
    .footer-note {
      margin-top: 2.5rem;
      border-top: 1px dashed #b9c8dd;
      padding-top: 1rem;
      font-size: 0.85rem;
      color: #556c83;
      text-align: center;
    }
    .footer-note code {
      background: #e8edf5;
      padding: 0.2rem 0.5rem;
      border-radius: 30px;
    }
    /* output screenshot simulation (simple frame) */
    .output-hint {
      background: #dee5f0;
      border-radius: 30px;
      padding: 0.25rem 1rem;
      margin-bottom: 1.4rem;
      display: inline-block;
      font-size: 0.8rem;
      color: #20344b;
    }
  </style>
</head>
<body>
  <div class="card">
    <h1>   TaskFlow</h1>
    <div class="subhead">employee · task manager</div>

    <!-- React mounts here -->
    <div id="root"></div>

    <div class="footer-note">
      ⚡ toggle changes status · <code>useState</code> live update<br>
         <strong>output screenshot (equivalent)</strong> — see list below
    </div>
  </div>

  <!-- Babel script: TaskList component + render -->
  <script type="text/babel">
    // TaskList component using React (FC)
    const TaskList = () => {
      // Initial tasks as given in requirement
      const [tasks, setTasks] = React.useState([
        { name: "Task 1", status: "done" },
        { name: "Task 2", status: "not done" },
        { name: "Task 3", status: "done" }
      ]);

      // Toggle function: maps over tasks, flips status of the selected one
      const toggleStatus = (indexToToggle) => {
        setTasks(prevTasks =>
          prevTasks.map((task, idx) => {
            if (idx === indexToToggle) {
              // toggle between "done" <-> "not done"
              const newStatus = task.status === "done" ? "not done" : "done";
              return { ...task, status: newStatus };
            }
            return task;
          })
        );
      };

      // Rendering the list
      return (
        <div>
          {/* subtle output hint to mimic screenshot description */}
          <div className="output-hint">
            🅸 current tasks (exactly as in sample + toggle)
          </div>
          <ul className="task-list">
            {tasks.map((task, index) => (
              <li key={index} className="task-item">
                <div className="task-info">
                  <span className="task-name">{task.name}</span>
                  <span className={`status-badge ${task.status === 'done' ? 'status-done' : 'status-not-done'}`}>
                    {task.status}
                  </span>
                </div>
                <button
                  className="toggle-btn"
                  onClick={() => toggleStatus(index)}
                >
                  ✓ toggle
                </button>
              </li>
            ))}
          </ul>
          {/* optional extra stats (counts) */}
          <div style={{ marginTop: '1.2rem', fontSize: '0.9rem', color: '#317f9b', background: '#eaf3fb', padding: '0.5rem 1.2rem', borderRadius: '50px' }}>
            ✅ done: {tasks.filter(t => t.status === 'done').length}  ·  ⏳ not done: {tasks.filter(t => t.status === 'not done').length}
          </div>
        </div>
      );
    };

    // Render TaskList into #root
    const rootElement = document.getElementById('root');
    ReactDOM.createRoot(rootElement).render(<TaskList />);
  </script>

  <!-- Additional note: this output can be taken as a screenshot replica.
       In a real PDF, you would capture the rendered area.
       For assignment purposes, the description below verifies the output. -->
  <div style="position: fixed; bottom: 10px; right: 10px; background: #ffffd0; padding: 0.3rem 1rem; border-radius: 40px; font-size: 0.75rem; border: 1px solid #e6e6b0;">
       output matches requirement: Task 1 (done), Task 2 (not done), Task 3 (done) + toggle
  </div>
</body>
</html>
```

How the Task Manager Works

You can interact with the list to update each task's completion status.

- **Task Display**: Each task shows its name and a colored badge indicating "done" or "not done".

- **Toggling Status**: Click the "toggle" button next to any task. This triggers the `toggleStatus` function, which updates the state and immediately flips the task's status badge.
- **Live Summary**: A footer counter shows the current number of "done" and "not done" tasks, updating with every toggle.